CAPSTONE PROJECT · PERTEMUAN 6

Implementasi Backend (REST API) & Database

Fase 1: Pengembangan Sistem · CPMK033

4 SKS · Tim 3 Orang · Project Based Learning

Fasilitator: Merry Anggraeni, S.Kom., M.Kom. | Capstone Project | FTI Universitas Budi Luhur

✅ Yang WAJIB Sudah Kelar di Pertemuan 6

Cek progres tim kamu: jika ada yang belum, KEJAR hari ini!

📋 Persiapan & Desain

- Tim 3 orang sudah fix & aktif
- Judul project sudah fix (tidak berubah lagi)
- Tech stack dipilih (frontend + backend + DB)
- WBS + jadwal (Gantt) sudah dibuat
- Desain UI/UX (wireframe) sederhana
- Desain database: ERD + DDL (tabel sudah dibuat)

⚙️ Implementasi Mulai

- Repo GitHub / project folder sudah disetup
- Struktur folder project rapi
- Koneksi database dari backend sudah jalan
- Minimal 1 endpoint API sudah berfungsi (GET)
- Ada data dummy di database (min. 5 baris)
- Pembagian tugas per anggota jelas
- Milestone 1 (akhir P8) sudah dipahami
- Tidak ada anggota tim yang nganggur

🗓️ Agenda Pertemuan 6 (4 SKS · ~200 menit)

0-15 mnt Cek progres tiap tim + laporan singkat

15-20 mnt Show & tell: 1-2 tim presentasi progress

20-60 mnt Materi + live demo: Backend REST API & Database

60-130 mnt Hands-on terbimbing: tiap tim bikin 2-3 endpoint

130-180 mnt Konsultasi per tim (rotasi bimbingan)

180-195 mnt Mini check-out: tulis progres & kendala

195-200 mnt Closing + set target Pertemuan 7

💡 **Fokus hari ini: PRAKTIK** **Capstone = Project Based Learning.**

🎯 Tujuan Pembelajaran

- Mengubah ERD menjadi tabel database yang benar (kunci, constraint, indeks)
- Menyepakati kontrak API lalu membangun endpoint inti
- Menerapkan validasi input dan kode status yang tepat
- Menguji endpoint dan mendokumentasikannya

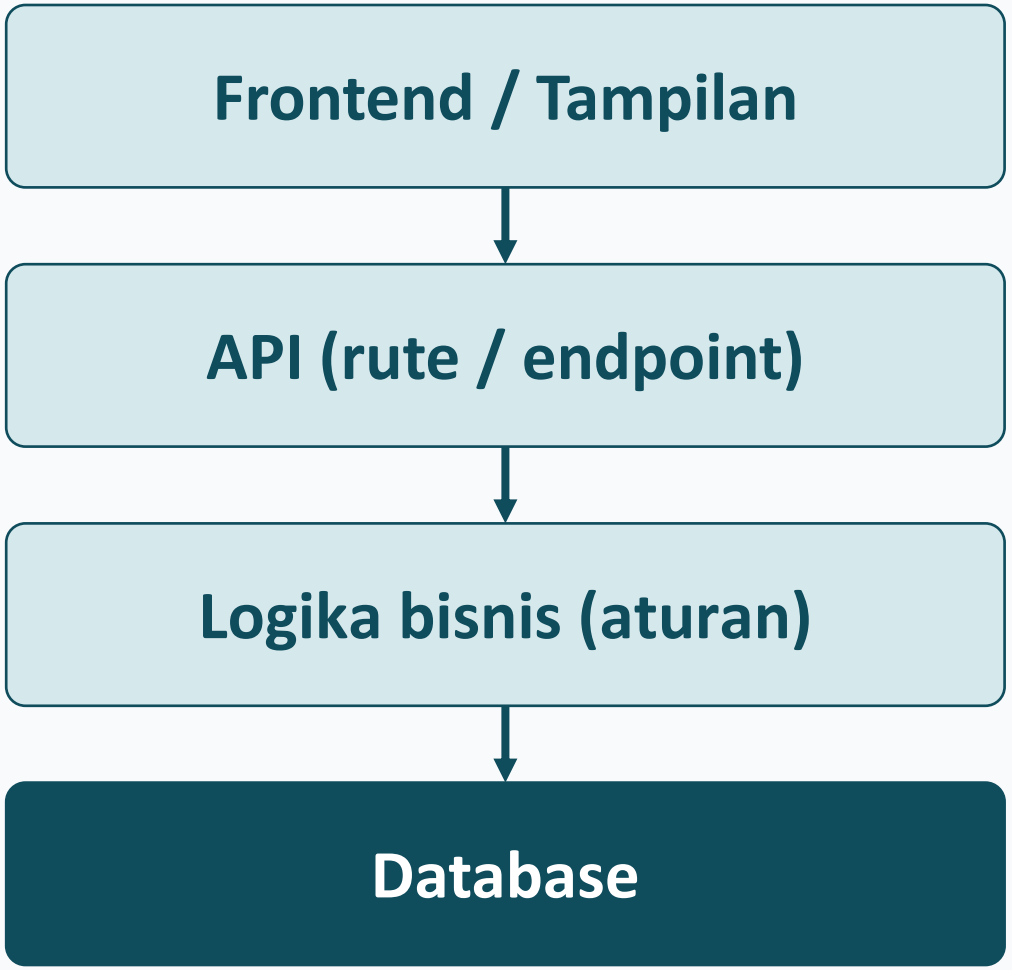
Apa itu REST API?

REST API = jembatan antara frontend (yang dilihat user) dengan backend (logika & database). Frontend minta data → API ambil dari database → kirim balik ke frontend.

GET	POST	PUT	DELETE
Ambil data	**Tambah data**	**Ubah data**	**Hapus data**
Lihat daftar / detail	Input / daftar baru	Edit / update data	Hapus record

Contoh: GET /api/laporan → ambil semua laporan | POST /api/laporan → tambah laporan baru

Arsitektur Berlapis: Siapa Mengerjakan Apa



Frontend: yang dilihat dan diklik pengguna (Pertemuan 7)

API: menerima permintaan, memvalidasi input, mengembalikan JSON + kode status

Logika: aturan sistem, mis. alur status laporan

Database: menyimpan data dan menjaga integritasnya

📜 Kontrak API & Kode Status HTTP

Metode	URL	Fungsi	Sukses
GET	/api/tugas	Daftar tugas	200
POST	/api/tugas	Tambah tugas (JSON: judul)	201 (400 jika salah)

Kode	Arti	Kapan dipakai
200 / 201	Berhasil / berhasil dibuat	Baca data / buat data baru
400	Input salah	Judul kosong, terlalu panjang
404	Tidak ditemukan	ID tidak ada
500	Error server	Bug di kode backend

Contoh Endpoint & Database (Studi Kasus: Si-RT/RW)

Endpoint CRUD Laporan:

- GET /api/laporan → ambil semua
- GET /api/laporan/{id} → ambil 1
- POST /api/laporan → tambah baru
- PUT /api/laporan/{id} → ubah
- DELETE /api/laporan/{id} → hapus

Tabel database (contoh):

```
CREATE TABLE laporan (
  id INT PRIMARY KEY AUTO_INCREMENT,
  judul VARCHAR(255),
  deskripsi TEXT,
  status ENUM('Baru','Proses','Selesai'),
  created_at TIMESTAMP DEFAULT NOW()
);
```

✅ **Target hari ini: MINIMAL 2 endpoint (GET + POST) terhubung ke database & bisa di-test (Postman / browser).**

Dari ERD ke Tabel SQL (contoh SiLapor RT)

```
CREATE TABLE IF NOT EXISTS laporan (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    pelapor_id INTEGER NOT NULL REFERENCES pengguna(id),
    kategori TEXT NOT NULL,
    jenis TEXT NOT NULL,
    judul TEXT NOT NULL,
    deskripsi TEXT NOT NULL,
    lokasi TEXT NOT NULL,
    status TEXT NOT NULL DEFAULT 'baru',
    dibuat TEXT NOT NULL DEFAULT CURRENT_TIMESTAMP
);
CREATE INDEX IF NOT EXISTS idx_laporan_status
    ON laporan(status);
```

PRIMARY KEY: identitas baris

REFERENCES (FK): relasi antartabel

NOT NULL / DEFAULT: data wajib & nilai awal

INDEX: cari/filter lebih cepat

🔒 Aman Sejak Awal: Query Berparameter & Validasi

Rentan: SQL disusun dari input pengguna

```
q = "SELECT * FROM tugas WHERE judul = '" + judul + "'"
db.execute(q)
```

Aman: query berparameter + validasi di server

```
judul = str(data.get("judul", "")).strip()
if not judul or len(judul) > 100:
    return jsonify(error="judul wajib, maks 100 karakter"), 400
db.execute("INSERT INTO tugas (judul) VALUES (?)", (judul,))
```

Aturan: jangan pernah menyambung string SQL dari input pengguna. Pakai tanda tanya (?) + parameter.



Kode Modul Inti (Flask) 1/3: Koneksi Database

```
import sqlite3
from flask import Flask, jsonify, request, g
app = Flask(__name__)
DB = "capstone.db"

def get_db():
    if "db" not in g:
        g.db = sqlite3.connect(DB)
        g.db.row_factory = sqlite3.Row
    return g.db

@app.teardown_appcontext
def close_db(_):
    if "db" in g:
        g.pop("db").close()
```

Kode Modul Inti (Flask) 2/3: Endpoint GET dan POST

```
@app.get("/api/tugas")
def list_tugas():
    q = "SELECT id, judul, selesai FROM tugas ORDER BY id DESC"
    return jsonify([dict(r) for r in get_db().execute(q)])

@app.post("/api/tugas")
def add_tugas():
    data = request.get_json(silent=True) or {}
    judul = str(data.get("judul", "")).strip()
    if not judul or len(judul) > 100:
        return jsonify(error="judul wajib, maks 100 karakter"), 400
    db = get_db()
    cur = db.execute("INSERT INTO tugas (judul) VALUES (?)", (judul,))
    db.commit()
    return jsonify(id=cur.lastrowid, judul=judul, selesai=0), 201
```

Kode Modul Inti (Flask) 3/3: Tabel & Menjalankan

```
def init_db():
    with sqlite3.connect(DB) as c:
        c.execute(
            "CREATE TABLE IF NOT EXISTS tugas ("
            "id INTEGER PRIMARY KEY AUTOINCREMENT, "
            "judul TEXT NOT NULL, selesai INTEGER DEFAULT 0)"
        )

if __name__ == "__main__":
    init_db()
    app.run(debug=True)  # matikan debug saat produksi
```

Jalankan: pip install flask → python app.py → buka http://127.0.0.1:5000/api/tugas

🐘 Alternatif Laravel/PHP 1/2: Controller

```
class LaporanController extends Controller {
    // GET /api/laporan — ambil semua data
    public function index() {
        return response()->json(Laporan::all(), 200);
    }

    // POST /api/laporan — tambah data baru
    public function store(Request $request) {
        $data = Laporan::create($request->all());
        return response()->json($data, 201);
    }

    // (lanjut: method show() di slide berikut)
}
```

⚠️ **Sebelum dipakai nyata: validasi input ($request->validate) & batasi kolom ($fillable). Kita bahas di Pertemuan 8.**

Alternatif Laravel/PHP 2/2: show() & Route

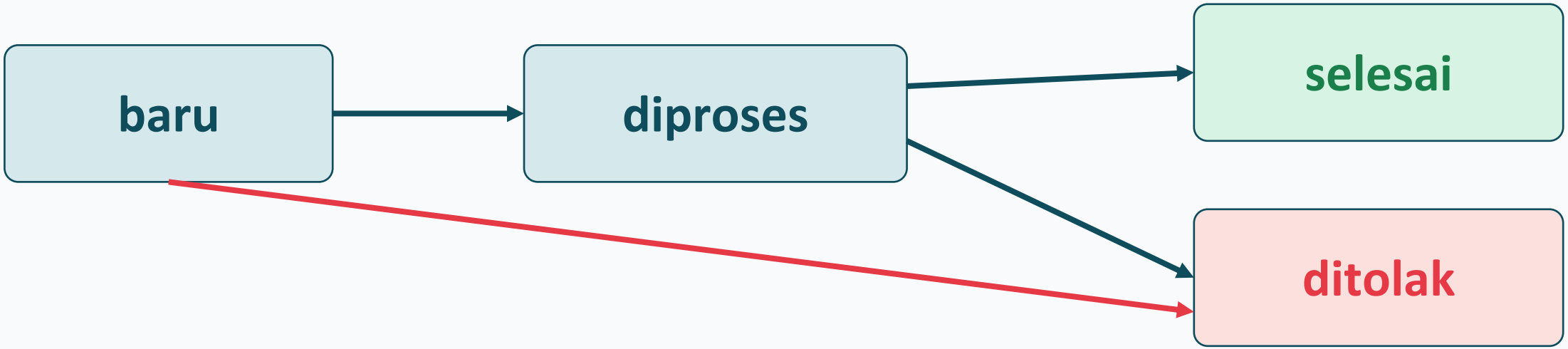
```
    // Tambahkan di dalam LaporanController yang sama

    // GET /api/laporan/{id} – ambil 1 data
    public function show($id) {
        return response()->json(Laporan::find($id), 200);
    }

// Route: api.php
Route::apiResource('laporan', LaporanController::class);
```

Uji dengan Postman atau browser: GET /api/laporan lalu POST /api/laporan.

🔄 Logika Bisnis: Alur Status Laporan




```
TRANSISI = {"baru": ["diproses", "ditolak"],
            "diproses": ["selesai", "ditolak"],
            "selesai": [], "ditolak": []}

if baru not in TRANSISI[lap["status"]]:
    flash("Perubahan status tidak diizinkan.")
```

🧪 Menguji Endpoint dengan pytest

```
import pytest
import app as m

@pytest.fixture
def client(tmp_path):
    m.DB = str(tmp_path / "uji.db")
    m.init_db()
    return m.app.test_client()

def test_tambah_dan_daftar(client):
    r = client.post("/api/tugas", json={"judul": "Buat ERD"})
    assert r.status_code == 201
    assert client.get("/api/tugas").json[0]["judul"] == "Buat ERD"

def test_judul_kosong_ditolak(client):
    assert client.post("/api/tugas", json={"judul": ""}).status_code == 400
```

Simpan test_api.py → python -m pytest -q → 2 passed

🛠️ Hands-on: Langkah Praktik Tim

Langkah	Kegiatan	Bukti
1	Buat tabel dari ERD timmu (schema.sql)	file schema.sql
2	Sepakati 3 endpoint inti di kontrak API	tabel endpoint di README
3	Implementasikan 1 endpoint GET dan 1 POST	kode di branch fitur
4	Uji (Postman / pytest), commit, buat pull request	hasil uji + PR

Tugas minggu depan: modul inti backend + database berjalan, endpoint terdokumentasi, minimal 3 tes lolos.

🎯 Target Pertemuan 7 (Seminggu Lagi!)

- Minimal 3 endpoint REST API berfungsi (GET all, GET by id, POST)
- Database terhubung + ada data dummy (min. 5 baris per tabel utama)
- Minimal 1 halaman frontend yang SUDAH memanggil API (integrasi mulai)
- Semua anggota tim punya kontribusi kode (cek commit GitHub!)

💬 **Konsultasi: grup WA · kelas · janji temu. Jangan malu bertanya! Semangat!** 💪

Checklist Selesai & Kesalahan Umum

Definition of Done

- Sesuai kontrak API
- Validasi input di server
- Query berparameter
- Ada tes otomatis
- Direview lewat pull request
- README diperbarui

Kesalahan umum

- SQL disusun dari string input
- Semua logika di satu fungsi besar
- Tidak ada penanganan error (selalu 200)
- Skema berubah tanpa memberi tahu tim
- Rahasia (password DB) masuk repo

Rangkuman & Acuan

- ERD → tabel SQL dengan kunci dan constraint
- Kontrak API disepakati dulu; endpoint mengembalikan JSON + kode status
- Validasi di server, query berparameter
- Logika bisnis (alur status) ditegakkan di backend
- Uji otomatis sebelum merge

Acuan (pustaka RPS): Jalote (2025); Mejía dkk. (2024); Valacich dkk. (2024); Schwalbe (2019). Pendukung: Suwarno dkk. (2025); Castillo & Monoso (2024).

Selamat Berkarya, Mahasiswa! 🚀

Hari ini = praktik nyata. Hasil hari ini = modal Milestone 1 (Pertemuan 8).

Merry Anggraeni, S.Kom., M.Kom. | FTI Universitas Budi Luhur